

Credit_card_fraud_detection

June 8, 2026

```
[25]: # Import libraries:

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, \
    accuracy_score
```

```
[26]: # Load dataset:

df = pd.read_csv("creditcard.csv")

df.head(10)
```

```
[26]:
```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	
5	2.0	-0.425966	0.960523	1.141109	-0.168252	0.420987	-0.029728	0.476201	
6	4.0	1.229658	0.141004	0.045371	1.202613	0.191881	0.272708	-0.005159	
7	7.0	-0.644269	1.417964	1.074380	-0.492199	0.948934	0.428118	1.120631	
8	7.0	-0.894286	0.286157	-0.113192	-0.271526	2.669599	3.721818	0.370145	
9	9.0	-0.338262	1.119593	1.044367	-0.222187	0.499361	-0.246761	0.651583	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	
5	0.260314	-0.568671	...	-0.208254	-0.559825	-0.026398	-0.371427	-0.232794	

```

6  0.081213  0.464960 ... -0.167716 -0.270710 -0.154104 -0.780055  0.750137
7 -3.807864  0.615375 ...  1.943465 -1.015455  0.057504 -0.649709 -0.415267
8  0.851084 -0.392048 ... -0.073425 -0.268092 -0.204233  1.011592  0.373205
9  0.069539 -0.736727 ... -0.246914 -0.633753 -0.120794 -0.385050 -0.069733

```

```

      V26      V27      V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724   2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0
5  0.105915  0.253844  0.081080    3.67      0
6 -0.257237  0.034507  0.005168    4.99      0
7 -0.051634 -1.206921 -1.085339   40.80      0
8 -0.384157  0.011747  0.142404   93.20      0
9  0.094199  0.246219  0.083076    3.68      0

```

[10 rows x 31 columns]

```
[27]: df.shape
```

```
[27]: (284807, 31)
```

```
[28]: # checking null values

df.isnull().sum().sum()
```

```
[28]: np.int64(0)
```

```
[29]: # Checking duplicates

df.duplicated().sum()
```

```
[29]: np.int64(1081)
```

```
[30]: df = df.drop_duplicates()
df
```

```
[30]:
```

	Time	V1	V2	V3	V4	V5	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	
...	...	...	...	...	...	...	
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	

```

284805 172788.0 -0.240440 0.530483 0.702510 0.689799 -0.377961
284806 172792.0 -0.533413 -0.189733 0.703337 -0.506271 -0.012546

```

```

          V6          V7          V8          V9  ...          V21          V22  \
0      0.462388  0.239599  0.098698  0.363787  ... -0.018307  0.277838
1     -0.082361 -0.078803  0.085102 -0.255425  ... -0.225775 -0.638672
2      1.800499  0.791461  0.247676 -1.514654  ...  0.247998  0.771679
3      1.247203  0.237609  0.377436 -1.387024  ... -0.108300  0.005274
4      0.095921  0.592941 -0.270533  0.817739  ... -0.009431  0.798278
...
284802 -2.606837 -4.918215  7.305334  1.914428  ...  0.213454  0.111864
284803  1.058415  0.024330  0.294869  0.584800  ...  0.214205  0.924384
284804  3.031260 -0.296827  0.708417  0.432454  ...  0.232045  0.578229
284805  0.623708 -0.686180  0.679145  0.392087  ...  0.265245  0.800049
284806 -0.649617  1.577006 -0.414650  0.486180  ...  0.261057  0.643078

          V23          V24          V25          V26          V27          V28  Amount  \
0     -0.110474  0.066928  0.128539 -0.189115  0.133558 -0.021053  149.62
1      0.101288 -0.339846  0.167170  0.125895 -0.008983  0.014724    2.69
2      0.909412 -0.689281 -0.327642 -0.139097 -0.055353 -0.059752  378.66
3     -0.190321 -1.175575  0.647376 -0.221929  0.062723  0.061458  123.50
4     -0.137458  0.141267 -0.206010  0.502292  0.219422  0.215153   69.99
...
284802  1.014480 -0.509348  1.436807  0.250034  0.943651  0.823731    0.77
284803  0.012463 -1.016226 -0.606624 -0.395255  0.068472 -0.053527   24.79
284804 -0.037501  0.640134  0.265745 -0.087371  0.004455 -0.026561   67.88
284805 -0.163298  0.123205 -0.569159  0.546668  0.108821  0.104533   10.00
284806  0.376777  0.008797 -0.473649 -0.818267 -0.002415  0.013649  217.00

```

```

Class
0      0
1      0
2      0
3      0
4      0
...
284802  0
284803  0
284804  0
284805  0
284806  0

```

```
[283726 rows x 31 columns]
```

```
[31]: df.shape
```

```
[31]: (283726, 31)
```

```
[32]: # Check fraud distribution
```

```
df['Class'].value_counts()
```

```
[32]: Class
```

```
0    283253
```

```
1      473
```

```
Name: count, dtype: int64
```

```
[33]: sns.countplot(x='Class', data=df)
plt.title("Fraud Vs Non-Fraud Transaction")
plt.show()
```



```
[34]: # Scale amount column
```

```
scaler = StandardScaler()
```

```
df['Amount'] = scaler.fit_transform(df[['Amount']])
```

```
df['Time'] = scaler.fit_transform(df[['Time']])
```

```
[35]: # Split feature and target

x = df.drop('Class', axis=1)
y = df['Class']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state = 42, stratify = y)
```

```
[36]: # Train logistic regression model

model = LogisticRegression(max_iter = 5000, random_state = 42)

model.fit(x_train, y_train)
```

```
[36]: LogisticRegression(max_iter=5000, random_state=42)
```

```
[37]: # Prediction

y_pred = model.predict(x_test)
```

```
[38]: # Evaluate model

print("Accuracy:" , accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy: 0.9991365030134283

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56651
1	0.85	0.59	0.70	95
accuracy			1.00	56746
macro avg	0.92	0.79	0.85	56746
weighted avg	1.00	1.00	1.00	56746

```
[39]: # Confusion matrix

cm = confusion_matrix(y_test, y_pred)

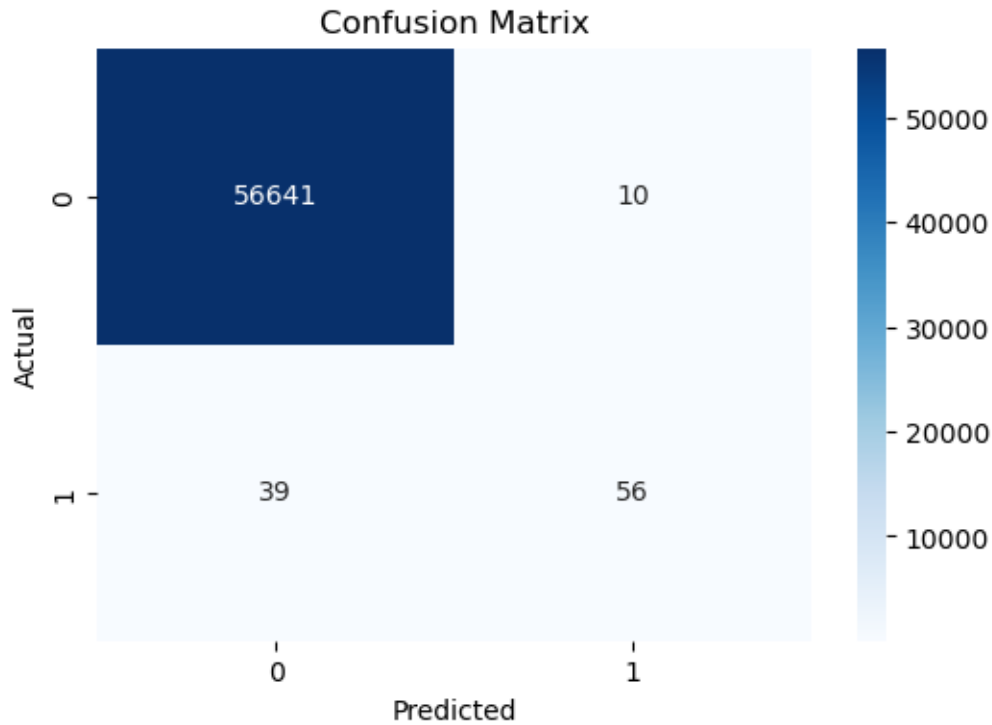
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
```

```

    fmt='d',
    cmap='Blues'
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



[40]: # Confusion matrix explanation:

```

# 1. Out of 56,651 genuine transactions, the model correctly classified 56,641
    ↳ of them.
# 2. Only 10 genuine transactions were wrongly marked as fraudulent.
# 3. The model missed 39 fraud cases.
# 4. The model successfully caught 56 fraudulent transactions.

# The model performed very well in identifying genuine transactions, correctly
    ↳ classifying
# 56,641 out of 56,651 legitimate transactions. It generated only 10 false
    ↳ alarms.
# For fraudulent transactions, the model successfully detected 56 fraud cases
    ↳ while missing

```

```
# 39. This indicates strong overall performance, although further improvements  
↳ can be made  
# to increase fraud detection recall and reduce missed fraud cases.
```

```
[41]: # Apply smote  
  
from imblearn.over_sampling import SMOTE  
  
smote = SMOTE(random_state=42)  
  
x_train_resampled, y_train_resampled = smote.fit_resample(  
    x_train, y_train  
)  
  
print("Before SMOTE:")  
print(y_train.value_counts())  
  
print("\nAfter SMOTE:")  
print(y_train_resampled.value_counts())
```

Before SMOTE:

Class

0 226602

1 378

Name: count, dtype: int64

After SMOTE:

Class

0 226602

1 226602

Name: count, dtype: int64

```
[42]: # Train Random forest model  
  
from sklearn.ensemble import RandomForestClassifier  
  
print("training started.....")  
rf_model = RandomForestClassifier(  
    n_estimators = 50,  
    random_state = 42,  
    n_jobs=-1  
)  
  
rf_model.fit(x_train_resampled, y_train_resampled)  
print("training completed")
```

training started..

training completed

```
[43]: # Make Predictions on Original Test Set
```

```
y_pred = rf_model.predict(x_test)
```

```
[44]: # Evaluate the Model
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
print(classification_report(y_test, y_pred))
```

```
print(confusion_matrix(y_test, y_pred))
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm,  
            annot=True,  
            fmt='d',  
            cmap='Blues')
```

```
plt.title("Random Forest Confusion Matrix")
```

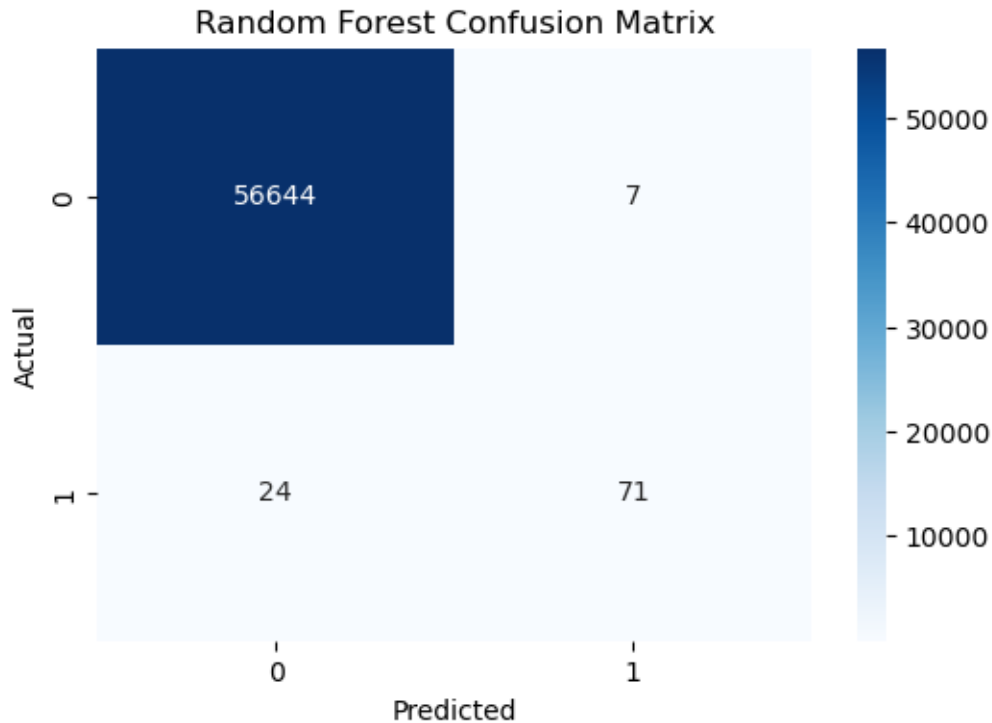
```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56651
1	0.91	0.75	0.82	95
accuracy			1.00	56746
macro avg	0.95	0.87	0.91	56746
weighted avg	1.00	1.00	1.00	56746

```
[[56644    7]  
 [   24   71]]
```

```
[45]: # Random forest confusion matrix explanation:  
  
# The Random Forest model performed exceptionally well, correctly classifying  
# 56,644  
# genuine transactions and detecting 71 fraudulent transactions. Only 7 genuine  
# transactions were incorrectly flagged as fraud, while 24 fraud cases were  
# missed.  
# This demonstrates strong fraud detection capability with a very low false  
# alarm rate.
```

```
[46]: # Accuracy  
  
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)
```

Accuracy: 0.9994537059880872

```
[47]: # Roc Auc score  
  
from sklearn.metrics import roc_auc_score  
  
y_prob = rf_model.predict_proba(x_test)[: , 1]
```

```
roc_auc = roc_auc_score(y_test, y_prob)

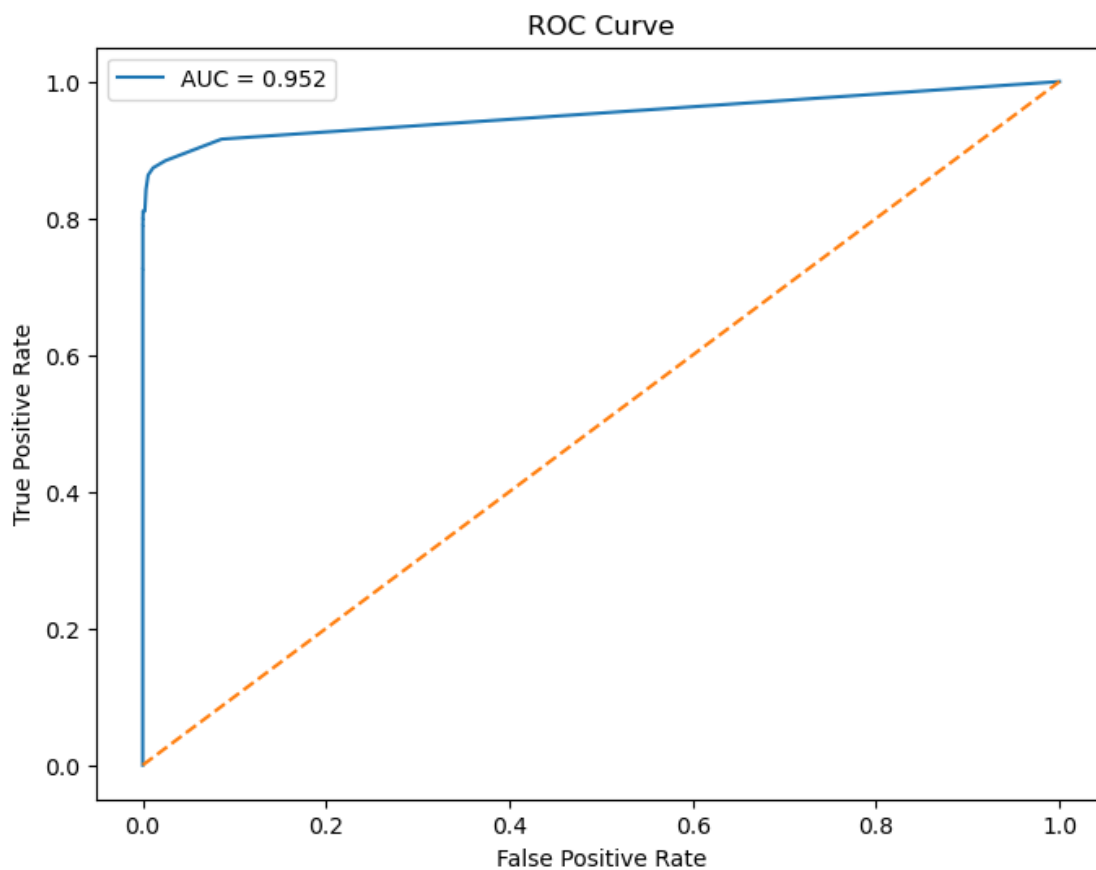
print("ROC-AUC Score:", roc_auc)
```

ROC-AUC Score: 0.9520263032473064

```
[48]: from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```



```
[ ]: # ROC-AUC curve explanation:

# The Random Forest model achieved a ROC-AUC score of 0.95, indicating
↳ excellent
# discrimination between fraudulent and genuine transactions. The ROC curve
↳ lies close
# to the top-left corner, showing that the model can identify fraudulent
↳ transactions with
# high accuracy while maintaining a low false positive rate.
```

```
[ ]: # Project conclusion:

# After applying SMOTE to address class imbalance, a Random Forest model was
↳ trained and
# evaluated. The model achieved 99.95% accuracy and a ROC-AUC score of 0.95.
↳ Compared to
# Logistic Regression, Random Forest significantly improved fraud detection
↳ recall
# from 59% to 75%, successfully identifying 71 out of 95 fraudulent
↳ transactions while
# generating only 7 false alarms. These results demonstrate that Random Forest
↳ with SMOTE
# is an effective approach for credit card fraud detection.
```

```
[49]: # Saving clean dataset for further Power BI analysis in .csv form

df.to_csv("creditcard_cleaned.csv", index=False)
```

```
[ ]:
```